

Introduction to Cloud Storage Services

Task 1

Create a new S3 bucket in your AWS account with a unique name that includes your initials and the current date (e.g., mybucket-AB-20240618). Set the region to Asia Pacific (Mumbai).

Steps followed:

1. Logged in to the AWS Management Console and opened the S3 service.
2. Clicked "Create bucket" and entered a globally unique bucket name including my initials and today's date: "mybucket-AB-20260809".
3. Set the AWS Region to "Asia Pacific (Mumbai) ap-south-1" from the region dropdown, so the bucket and its objects are stored closer to users in India.
4. Left Object Ownership at the default (ACLs disabled, bucket owner enforced) and kept "Block all public access" enabled at this stage, since specific objects would be made public individually later in Task 3.
5. Left default encryption enabled (SSE-S3) and bucket versioning left off for now, since it would be turned on separately in Task 4.
6. Reviewed the settings and clicked "Create bucket". Confirmed the new bucket "mybucket-AB-20260809" appeared in the bucket list with the region shown as "ap-south-1".

Task 2

Upload three different files (an image, a PDF, and a text file) to your new S3 bucket using the AWS Management Console. Make sure each file has a meaningful name related to your favorite app (e.g., instagram-logo.png, spotify-playlist.pdf, zomato-menu.txt).

Steps followed:

1. Opened the newly created bucket and clicked "Upload".
2. Added an image file named "instagram-logo.png" (a small PNG saved locally) using "Add files".
3. Added a PDF file named "spotify-playlist.pdf" (a short PDF exported with a sample playlist list) to the same upload batch.
4. Added a plain text file named "zomato-menu.txt" containing a short sample restaurant menu.
5. Left storage class as "Standard" for all three files and clicked "Upload".
6. Once the upload completed, refreshed the bucket's Objects tab and confirmed all three files — instagram-logo.png, spotify-playlist.pdf, and zomato-menu.txt — were listed with their correct sizes and "Standard" storage class.

Task 3

Set the access permissions on one of your uploaded files so it is publicly accessible via a URL. Test the link in your browser to confirm it works.

Steps followed:

1. Selected "instagram-logo.png" in the bucket and opened its "Permissions" tab.
2. Since the bucket-level "Block all public access" setting blocks any object ACL/policy from taking effect, first went to the bucket's Permissions tab and edited "Block public access" to uncheck it for this bucket, confirming the warning prompt.

3. Went back to the object, clicked "Object actions" → "Make public using ACL", and confirmed the action in the dialog that appeared.
4. Copied the object's URL from the "Object URL" field on the object's Properties tab (in the form `https://mybucket-AB-20260809.s3.ap-south-1.amazonaws.com/instagram-logo.png`).
5. Pasted the URL into a browser in a new tab and confirmed the image loaded directly without needing to be logged into AWS, verifying the file was genuinely publicly accessible.
6. Left the other two files (`spotify-playlist.pdf` and `zomato-menu.txt`) private, so only this one object was exposed publicly, keeping the rest of the bucket protected.

Task 4

Enable versioning on your S3 bucket, then upload a new version of one of your files (e.g., upload an updated `zomato-menu.txt` with different content). Check the version history to confirm both versions are stored.

Steps followed:

1. Opened the bucket's "Properties" tab and found the "Bucket Versioning" section.
2. Clicked "Edit", selected "Enable", and saved the change, turning on versioning for the entire bucket.
3. Edited the local `zomato-menu.txt` file to add a couple of new menu items, changing its content from the original version.
4. Uploaded this updated file to the same bucket using the same key name "`zomato-menu.txt`", which S3 treats as a new version rather than an overwrite once versioning is enabled.
5. Opened the object "`zomato-menu.txt`" and switched on "Show versions" (the Versions tab), which listed two entries with different Version IDs — the original upload and the newer updated one, each with its own "Last modified" timestamp.
6. Downloaded both versions individually using the "Version ID" links to confirm the older version still contained the original menu text and the latest version contained the updated content, proving both copies were retained rather than the old one being lost.

Task 5

Write a short comparison (3-4 sentences) explaining the main differences between AWS S3, EBS, and Glacier. Mention which one you would use for storing Instagram profile pictures, Flipkart order history, and old IRCTC booking records, and why.

Amazon S3 is object storage accessed over the web (via URLs/APIs), designed for storing and serving large numbers of independent files such as images, documents, and backups with high durability and easy public/private access control. Amazon EBS is block storage that attaches directly to a single EC2 instance like a virtual hard disk, used for data that an application needs fast, low-latency, file-system-style read/write access to, such as a database's live data files — it is not meant to be accessed by multiple servers or over the internet the way S3 objects are. Amazon Glacier (S3 Glacier) is a low-cost archival storage class built for data that is rarely accessed and can tolerate a longer retrieval time in exchange for a much lower storage price, making it suited for long-term backups and compliance records rather than everyday access.

For Instagram profile pictures, I would use S3, since they are individual image files that need to be quickly and publicly retrievable by URL for millions of users at once. For Flipkart order history, I would also use S3 (or a database backed by EBS for the live transactional records, with older completed orders periodically moved to S3), since order records need to remain readily accessible whenever a customer looks them up, but don't need the raw-disk performance that EBS is built for. For old IRCTC booking records that are rarely looked at once a journey is completed, I would use Glacier, since it is the cheapest option for data that mostly just needs to be retained for record-keeping rather than accessed frequently.